ENN in Lean 4

Written by Codex and edited by Mehul Bafna.

Abstract

We formalize Appendix C of *Taking the GP Out of the Loop* [1] in Lean 4.

Formal system

The formalization begins with finite weighted averages and proves approximation, consistency, and parameter-stability results for ENN. The exact-observation results concern continuous functions on metric spaces; the noisy results concern conditional moments and limits under stated sampling assumptions. The fitted-parameter results separate stability of the predictor from convergence of the fitting criterion. The corresponding declarations and their relation to Appendix C are given below.

Lean 4 checks proofs of these mathematical statements [2]. Mathlib supplies the underlying results on finite sums, topology, conditional expectation, and measurability [3, 4]; the appendix identifies their use at the level of individual declarations.

Organization

The source follows the progression from weights and ordered observations to prediction, noise, and fitted parameters. Captions below the mathematical statements identify the relevant Lean source lines.

```
ENN.lean                        complete library import
ENN/
|-- FiniteWeights.lean          normalized finite weights
|-- InverseSquare.lean          inverse-square algebra
|-- NearestNeighbor.lean        ordered neighbors, radii, SNEB
|-- ExactObservation.lean       exact mean and consistency
|-- AcquisitionGeometry.lean    scalarization and improvement
|-- Noise/
|   |-- Estimator.lean          bias/noise decomposition
|   |-- Moments.lean            conditional moment expansion
|   |-- Bounds.lean             conditional MSE inequality
|   |-- Independence.lean       conditional orthogonality
|   |-- RandomDesign.lean       measurable random neighbors
|   |-- Consistency.lean        mean-square and probability limits
|   `-- Rates.lean              asymptotic rate algebra
|-- Fit/
|   |-- Box.lean                compact parameter domain
|   |-- Stability.lean          weight and predictor stability
|   |-- Argmax.lean             argmax convergence
|   |-- Transfer.lean           fitted-predictor consistency
|   `-- Selection.lean          measurable fitted predictor
|-- Select/
|   |-- Value.lean              measurable maximum value
|   |-- Cover.lean              measurable finite choices
|   |-- Step.lean               maximizing child selection
|   |-- Path.lean               nested maximizing cells
|   |-- Limit.lean              exact measurable maximizer
|   |-- Approx.lean             approximate selectors
|   `-- Unique.lean             unique-maximizer specialization
|-- PseudoBO/                   convergence assumptions and counterexample
`-- Paper/                      Appendix C theorem aliases
```

Exact-observation ENN

Definition

Let X be a metric space and $D : \mathrm{Fin}(n) \to X$ an indexed dataset. For a query x, sample i has lexicographic key $(d(x, D_i), i)$. The index component resolves distance ties. The first $K_n = \min(K, n)$ samples in this order form the neighbor prefix. Number these neighbors $z_1, \ldots, z_{K_n}$ in the mathematical notation below; Lean uses indices starting at zero. For an unobserved query and $K_n > 0$, all radii $r_i = d(x, z_i)$ are positive. Appendix C.1 defines

$$w_i(x) = \frac{r_i^{-2}}{\sum_{j=1}^{K_n} r_j^{-2}},$$

$$\mu_n(x) = \sum_{i=1}^{K_n} w_i(x) f(z_i),$$

$$\sigma_n(x) = \left(\sum_{i=1}^{K_n} r_i^{-2} \right)^{-1/2}.$$

This is equation (4) of the paper [1, Appendix C.1, Eq. (4)]. At an observed input, `ENN.Exact.mu` returns $f(x)$ and `ENN.Exact.sigma` returns 0. The definitions are total when $K_n = 0$; the inverse-distance theorems assume $K_n > 0$.

ENN/ExactObservation.lean:19--29

ENN/NearestNeighbor.lean:344--356

```
noncomputable def Exact.mu
    (data : ExactDataset (X := X) (n := n))
    (f : X → ℝ) (query : X) (K : ℕ) : ℝ

noncomputable def Exact.sigma
    (data : ExactDataset (X := X) (n := n))
    (query : X) (K : ℕ) : ℝ
```

Deterministic approximation

The prediction is a convex combination of observed function values. Its error is therefore bounded by the same weighted average of their errors. For an unsampled query with $K_n > 0$, let ω be monotone and suppose each retained neighbor satisfies $|f(z_i) - f(x)| \le \omega(r_i)$. Normalization and nonnegativity of the weights give

$$|\mu_n(x) - f(x)| \le \sum_{i=1}^{K_n} w_i(x) \omega(r_i) \le \omega(r_{K_n}).$$

This is the approximation argument in Theorem 1(i) [1, Appendix C.1, Theorem 1(i), Eq. (5)]. The formal theorem requires only the displayed pointwise bound and monotonicity of ω; a modulus of continuity is one choice satisfying those assumptions. In the declaration, `herr` is the pointwise bound and `hω` is monotonicity.

ENN/ExactObservation.lean:50--77

```
theorem Exact.error_bound
    (data : ExactDataset (X := X) (n := n)) (f : X → ℝ)
    (query : X) (K : ℕ) (hk : 0 < Neighbor.count K n)
```

```
    (hquery : ¬ Sampled data query) (ω : ℝ → ℝ)
    (herr : ∀ i : Fin (Neighbor.count K n),
      |f (data (Neighbor.prefix_index data query
        (Nat.min_le_right K n) i)) - f query| ≤
      ω (Neighbor.prefix_radius data query
        (Nat.min_le_right K n) i))
    (hω : Monotone ω) :
    |Exact.mu data f query K - f query| ≤
      ω (Neighbor.last_radius data query K hk)
```

Local consistency

Theorem 1(ii) states the spacing equivalence in equation (7) [1, Appendix C.1, Theorem 1(ii), Eq. (7)]. For a nonempty dataset and $K_n > 0$, let $d_n(x) = \min_i d(x, D_i) = r_1$. At an unsampled query, each inverse-square term is at most $d_n(x)^{-2}$, and the nearest neighbor contributes exactly that amount. Taking the reciprocal square root of the sum gives

$$\frac{d_n(x)}{\sqrt{K_n}} \le \sigma_n(x) \le d_n(x).$$

The inverse-square algebra is proved in `ENN.InvSq.spacing`; the ordered dataset specialization is `ENN.Exact.spacing`. At a sampled query both sides are zero. If every sample is at distance at least $r > 0$, `ENN.Exact.empty_ball` gives $\sigma_n(x) \ge r/\sqrt{K_n}$.

ENN/NearestNeighbor.lean:376--431

Theorem 1(iii–iv) states local consistency and sequential no-empty-ball [1, Appendix C.1, Theorem 1(iii–iv)]. Fix $K > 0$. For datasets D_m, suppose $x_m \to x_\star$ and one selected current sample obeys $d(x_m, D_m(i_m)) \to 0$. On a compact metric space, continuity of f implies

$$\mu_m(x_m) \to f(x_\star), \qquad \sigma_m(x_m) \to 0.$$

Only one sample needs to approach the query. To see why, split the weighted error into neighbors inside and outside a fixed small radius. Continuity controls the function values inside the radius. Outside it, each inverse-square term is bounded, while the nearest sample makes the normalization sum diverge. Since K is fixed, the total weight outside the radius tends to zero. Compactness bounds the function's range, so this vanishing weight also makes the distant observations' contribution vanish. Queries that coincide with observations are handled by exact interpolation. The uncertainty limit follows from the upper spacing bound.

ENN/ExactObservation.lean:287--506

ENN/NearestNeighbor.lean:468--481

```
theorem Exact.local_consistency [CompactSpace X]
    {nSeq : ℕ → ℕ}
    (data : ∀ m, ExactDataset (X := X) (n := nSeq m))
    (f : X → ℝ) (query : ℕ → X) {x' : X}
    (K : ℕ) (hK : 0 < K)
    (hq : Filter.Tendsto query Filter.atTop (nhds x'))
    (i : ∀ m, Fin (nSeq m))
    (hd : Filter.Tendsto
      (fun m ↦ dist (query m) (data m (i m)))
      Filter.atTop (nhds 0))
    (hf : Continuous f) :
    Filter.Tendsto
      (fun m ↦ Exact.mu (data m) f (query m) K)
      Filter.atTop (nhds (f x'))
```

Here `hq` describes the query limit, `hd` supplies the approaching sample, and `hf` supplies continuity. The dataset size may vary with m; the chosen neighbor count K does not. The proof uses the metric convergence and compactness lemmas cited in the mathlib appendix.

Acquisition geometry

Appendix C.2 defines the auxiliary scalarization and its positive-linear form in equations (15)–(16) [1, Appendix C.2, Eqs. (15)–(16)]. For $0 < \lambda < 1$, define

$$A_\lambda(g, u) = \lambda g^+ + (1-\lambda)u, \qquad g^+ = \max(g, 0).$$

Suppose $m \le \mu(x)$ for every feasible x. The auxiliary acquisition then equals $\lambda\mu(x) + (1-\lambda)\sigma(x) - \lambda m$. Maximization is therefore equivalent to maximizing a positive linear combination of (μ, σ). A point improving either coordinate without decreasing the other would strictly increase this score. Consequently, every maximizer is nondominated.

ENN/AcquisitionGeometry.lean:20--31,53--135

Proposition 1 states the two improvement clauses [1, Appendix C.2, Proposition 1]:

1. if u_n is eventually bounded below by $b > 0$, then the score is eventually bounded below by $(1-\lambda)b > 0$;
2. if $u_n \ge 0$, $u_n \to 0$, g_n is bounded above, and $\limsup_n g_n \le 0$, then $A_\lambda(g_n, u_n) \to 0$.

ENN/AcquisitionGeometry.lean:137--200

```
theorem Acq.vanishes
    {g u : ℕ → ℝ} {lam : ℝ}
    (hlam0 : 0 < lam) (hlam1 : lam < 1)
    (hu_nonneg : ∀ n, 0 ≤ u n)
    (hg_bdd : Filter.IsBoundedUnder LE.le Filter.atTop g)
    (hg : Filter.limsup g Filter.atTop ≤ 0)
    (hu : Filter.Tendsto u Filter.atTop (nhds 0)) :
    Filter.Tendsto
      (fun n ↦ Acq.score lam (g n) (u n))
      Filter.atTop (nhds 0)
```

The second clause needs control of the gain as well as the uncertainty: a positive gain can keep the acquisition positive even when uncertainty vanishes. This condition also appears in the formalization of the PseudoBO framework of Chen and Lam [6]. The local definition of evaluation worthiness requires the acquisition to tend to zero along moving queries approaching the data. The theorem `ENN.PseudoBO.worthy_of_gain` states the implication

$$\begin{aligned} &\text{Improvement}(A) \wedge \text{SNEB}(\sigma) \\ &\quad \wedge \text{VanishingGain}(g) \\ &\Longrightarrow \text{Worthwhile}(A_\lambda(g, \sigma)). \end{aligned}$$

The gain premise is explicit. `ENN.PseudoBO.baseline_counterexample` shows that the baseline $m_n = \inf_z \mu_n(z)$ alone does not imply worthiness for an arbitrary surrogate pair.

ENN/PseudoBO/Axioms.lean:49--105,130--143

ENN/PseudoBO/Counterexamples.lean:17--40

Noisy observations

Conditional MSE

Theorem 2 states the weighted-kNN conditional bias–variance inequality in equation (17) [1, Appendix C.3, Theorem 2, Eq. (17)]. Work on a probability space Ω with a conditioning σ-algebra m. First fix observation locations z_i and a query x. For a finite index set I, let $Y_i = f(z_i) + \varepsilon_i$ and $\tilde{\mu} = \sum_i w_i Y_i$, where the m-measurable weights are nonnegative and sum to one. Assume

$$\begin{aligned} \mathbb{E}[\varepsilon_i \mid m] &= 0, \\ \mathbb{E}[\varepsilon_i \varepsilon_j \mid m] &= 0 \quad (i \neq j), \\ \mathbb{E}[\varepsilon_i^2 \mid m] &\le \bar{\sigma}^2. \end{aligned}$$

Each noise is square-integrable. If $L \ge 0$, $|f(z_i) - f(x)| \le L d(z_i, x)$ and $d(z_i, x) \le \rho$, then, almost surely,

$$\mathbb{E}[(\tilde{\mu} - f(x))^2 \mid m] \le (L\rho)^2 + \bar{\sigma}^2 \sum_i w_i^2.$$

The first term bounds the error from using nearby function values; the second measures how much noise remains after averaging. The formal statement below allows a finite measure rather than requiring its total mass to be one. Its locations are deterministic, although its weights and noises are random. Random locations are treated separately by `ENN.Noise.mse_random`.

ENN/Noise/Bounds.lean:107--212

```
theorem Noise.mse
    [DecidableEq ι] {X : Type*} [MetricSpace X]
    (hm : m ≤ mΩ)
    (w : ι → Ω → ℝ) (f : X → ℝ) (z : ι → X)
    (ε : ι → Ω → ℝ) (x : X) {L ρ σ2 : ℝ}
    (hw_meas : ∀ i, StronglyMeasurable[m] (w i))
    (hw0 : ∀ i, ∀ᵐ ω ∂μ, 0 ≤ w i ω)
    (hw_sum : ∀ᵐ ω ∂μ, ∑ i, w i ω = 1)
    (hL : 0 ≤ L)
    (hLip : ∀ i, |f (z i) - f x| ≤ L * dist (z i) x)
    (hρ : ∀ i, dist (z i) x ≤ ρ)
    (hε2 : ∀ i, MemLp (ε i) 2 μ)
    (hε_zero : ∀ i, μ[ε i | m] =ᵐ[μ] 0)
    (hε_orth : ∀ i j, i ≠ j →
      μ[fun ω ↦ ε i ω * ε j ω | m] =ᵐ[μ] 0)
    (hε_sq : ∀ i,
      μ[fun ω ↦ (ε i ω) ^ 2 | m]
        ≤ᵐ[μ] fun _ ↦ σ2) :
    μ[fun ω ↦
      (∑ i, w i ω * (f (z i) + ε i ω) - f x) ^ 2 | m]
      ≤ᵐ[μ] fun ω ↦
        (L * ρ) ^ 2 + σ2 * ∑ i, (w i ω) ^ 2
```

The symbols $=^{\text{a.e.}}_{\mu}$ and $\le^{\text{a.e.}}_{\mu}$ are Lean's almost-everywhere equality and inequality; the bracket notation is conditional expectation.

Error decomposition

Write the prediction error as $B + N$, where $B = \sum_i w_i (f(z_i) - f(x))$ and $N = \sum_i w_i \varepsilon_i$. The bias B is determined by the information in m and satisfies $|B| \le L\rho$. The noise N has conditional mean zero. Thus the cross term in $(B + N)^2$ has conditional expectation zero. Expanding N^2 leaves only its diagonal terms, because distinct noises are conditionally orthogonal; bounding each conditional second moment gives the stated inequality.

The decomposition is `ENN.Estimator.decompose`. The bias and centering steps are `ENN.Noise.bias` and `ENN.Noise.noise_mean`; the square expansions are `ENN.Noise.noise_var_eq` and `ENN.Noise.sq_add_zero`.

ENN/Noise/Estimator.lean:22--70

ENN/Noise/Moments.lean:18--49,353--517

The paper assumes conditional independence. The formal MSE theorem uses the weaker pairwise conditional-orthogonality equations. `ENN.Noise.condExp_mul` derives conditional product factorization from mathlib's `CondIndepFun` on a standard Borel sample space, with measurable noises and integrable products [4]; conditional centering then proves `ENN.Noise.orthogonal`. Substituting this conclusion into the MSE bound proves `ENN.Noise.mse_indep`. Conditional independence is therefore sufficient, but the bound itself needs only the cross-moment equations.

ENN/Noise/Independence.lean:24--73

ENN/Noise/RandomDesign.lean:202--254

Consistency rates

The variance and normalized weights are those used in Appendix C.3 [1, Appendix C.3, definition preceding Theorem 2]:

$$\begin{aligned} v_i &= \alpha + s_i^2 + \gamma r_i^2, \\ w_i &= \frac{v_i^{-1}}{\sum_j v_j^{-1}}, \qquad \alpha > 0,\ \gamma \ge 0. \end{aligned}$$

The library proves positivity and normalization before applying the generic bound. `ENN.Noise.mse_ordered` uses the deterministic sorted dataset prefix. `ENN.Noise.mse_random` accepts measurable random neighbors and a square-integrable random radius.

ENN/Noise/Estimator.lean:107--158

ENN/OrderedWeightedKNN.lean:25--119

ENN/Noise/RandomDesign.lean:24--199

After integration, put $R_n = \mathbb{E}[\rho_n^2]$ and $W_n = \mathbb{E}[\sum_i w_{n,i}^2]$. `ENN.Noise.consistency_cond` proves that $R_n \to 0$ and $W_n \to 0$ imply mean-square convergence and convergence in probability. The first limit controls approximation error; the second controls noise. Neither follows from normalization of the weights alone. Integrating the conditional bound proves mean-square convergence, and Markov's inequality applied to the squared error proves convergence in probability. More quantitatively, assuming $R_n \lesssim a_n^2$ and $W_n \lesssim k_n^{-1}$ gives

$$\mathrm{MSE}_n \lesssim a_n^2 + k_n^{-1}.$$

The balanced choice $a_n \asymp (k_n/n)^{1/q}$ and $k_n \asymp n^{2/(q+2)}$ gives $\mathrm{MSE}_n \lesssim n^{-2/(q+2)}$. These are the consistency and

rate conclusions of Theorem 2 [1, Appendix C.3, Theorem 2(i–ii), Eq. (18)]. These hypotheses isolate the analytic estimates needed for the weighted estimator. Stone's consistency theorem provides classical context for weighted nonparametric regression [5].

ENN/Noise/Consistency.lean:124--148

```
theorem Noise.consistency_cond [IsFiniteMeasure μ]
    {m : MeasurableSpace Ω} (hm : m ≤ mₒ)
    (X : ℕ → Ω → ℝ) (target : Ω → ℝ)
    (rhs : ℕ → Ω → ℝ) (R W : ℕ → ℝ) {L σ2 : ℝ}
    (hsq_int : ∀ n, Integrable
      (fun ω ↦ (X n ω - target ω) ^ 2) μ)
    (hrhs_int : ∀ n, Integrable (rhs n) μ)
    (hcond : ∀ n, @condExp Ω ℝ m mₒ _ _ μ
      (fun ω ↦ (X n ω - target ω) ^ 2) ≤ᵐ[μ] rhs n)
    (hrhs_le : ∀ n, ∫ ω, rhs n ω ∂μ ≤
      L ^ 2 * R n + σ2 * W n)
    (hradius : Filter.Tendsto R Filter.atTop (nhds 0))
    (hweight : Filter.Tendsto W Filter.atTop (nhds 0)) :
    @MeanSquare Ω mₒ X target μ ∧
      @TendstoInMeasure Ω ℕ ℝ _ mₒ μ X Filter.atTop target
```

ENN/Noise/Rates.lean:39--130

```
theorem Noise.rate
    {mse R W rRate invK : ℕ → ℝ} {L σ2 : ℝ}
    (hσ2 : 0 ≤ σ2)
    (hmse_le : ∀ n, mse n ≤ L ^ 2 * R n + σ2 * W n)
    (hradius : RateBound R rRate)
    (hweight : RateBound W invK)
    (hrate0 : ∀ᶠ n in Filter.atTop, 0 ≤ rRate n)
    (hinv0 : ∀ᶠ n in Filter.atTop, 0 ≤ invK n) :
    RateBound mse (fun n ↦ rRate n + invK n)
```

Fitted parameters

The fitted parameter is $\phi = (\alpha, \gamma)$ in

$$\Phi = [\underline{\alpha}, \overline{\alpha}] \times [\underline{\gamma}, \overline{\gamma}], \qquad 0 < \underline{\alpha} \le \overline{\alpha}, \quad 0 \le \underline{\gamma} \le \overline{\gamma}.$$

For a nonempty finite neighbor set, $\phi, \psi \in \Phi$, and bounds $s_i^2 \le S^2$ and $r_i^2 \le R^2$, define

$$\Lambda = 2(\overline{\alpha} + S^2 + \overline{\gamma} R^2)\underline{\alpha}^{-2}(1 + R^2).$$

Then `ENN.Fit.weights_l1` and `ENN.Fit.predictor_l1` prove

$$\sum_i |w_i(\phi) - w_i(\psi)| \le \Lambda \|\phi - \psi\|_1,$$

$$|\tilde{\mu}(\phi) - \tilde{\mu}(\psi)| \le Y \Lambda \|\phi - \psi\|_1.$$

when $|y_i| \le Y$. The positive lower bound on α keeps every variance away from zero, so its reciprocal changes at most linearly with the parameters. The upper variance bound also keeps the normalization sum away from zero. Together these estimates control the normalized weights. The predictor bound then follows by multiplying each weight difference by its response and summing. These are equations (19) and (20) of Theorem 3 [1, Appendix C.4, Theorem 3, Eqs. (19)–(20)].

ENN/Fit/Stability.lean:327--433

Argmax consistency

The first conclusion of Theorem 3 is argmax consistency under uniform convergence and separation [1, Appendix C.4, Theorem 3]. Let L_n be an empirical criterion, Q_n its target, and $\hat{\phi}_n$ a maximizer of L_n, with $\phi_n^\star$ a reference point for Q_n. Let Δ_n bound $|L_n - Q_n|$ uniformly. A target is separated at (δ, η) if every point at distance at least δ loses at least η in target value relative to $\phi_n^\star$.

The argument is deterministic before it is probabilistic. Maximality of $\hat{\phi}_n$ for L_n, together with the two approximation errors at $\hat{\phi}_n$ and $\phi_n^\star$, bounds the loss in Q_n by $2\Delta_n$. Separation rules out a distance of at least δ whenever $\Delta_n < \eta/3$. Hence a large parameter error requires either failure of separation or a large uniform approximation error. Bounding the measure of their union proves the limit below.

ENN/Fit/Argmax.lean:40--90

```
theorem Fit.argmax_limit
    [IsFiniteMeasure μ]
    (distance : P → P → ℝ)
    (L Q : ℕ → Ω → P → ℝ)
    (hat star : ℕ → Ω → P) (Δ : ℕ → Ω → ℝ)
    (hhat : ∀ n ω p, L n ω p ≤ L n ω (hat n ω))
    (hclose : ∀ n ω p,
      |L n ω p - Q n ω p| ≤ Δ n ω)
    (huniform : ∀ ε, 0 < ε → Filter.Tendsto
      (fun n ↦ μ {ω | ε ≤ Δ n ω}) Filter.atTop (nhds 0))
    (hsep_prob : ∀ δ, 0 < δ → ∃ η, 0 < η ∧ Filter.Tendsto
      (fun n ↦ μ {ω | ¬ Separated distance
        (Q n ω) (star n ω) δ η}) Filter.atTop (nhds 0)) :
    ∀ δ, 0 < δ → Filter.Tendsto
      (fun n ↦ μ {ω | δ ≤ distance
        (hat n ω) (star n ω)}) Filter.atTop (nhds 0)
```

The declaration assumes empirical maximality through `hhat`. The comparison with the reference point is contained in `hsep_prob`. The theorem is stated as a measure-convergence result for random parameters.

Fitted consistency

Parameter convergence matters for prediction through the stability bound. Let the oracle predictor use the reference parameters $\phi_n^\star$, and suppose

$$|\tilde{\mu}_n^{\mathrm{fit}} - \tilde{\mu}_n^{\mathrm{or}}| \le \Gamma_n \|\hat{\phi}_n - \phi_n^\star\|_1,$$

If $\Gamma_n = O_{\mathbb{P}}(1)$ and the parameter error is $o_{\mathbb{P}}(1)$, the fitted and oracle predictions differ by a quantity tending to zero in probability. The triangle inequality then proves consistency of the fitted predictor whenever the oracle predictor is consistent. This is `ENN.Fit.fitted_prob`, corresponding to the probability conclusion of Theorem 3 [1, Appendix C.4, Theorem 3]. For mean-square consistency, `ENN.Fit.fitted_l2` assumes

$$\mathbb{E}[\Gamma_n^2 \|\hat{\phi}_n - \phi_n^\star\|_1^2] \to 0.$$

along with integrability of the relevant squared errors and mean-square consistency of the oracle. This stronger moment assumption controls the contribution of rare, large parameter errors.

ENN/Fit/Transfer.lean:133--164

Measurable selection

Theorem 3 uses random maximizers $\hat{\phi}_n$ and $\phi_n^\star$ in a convergence in probability statement [1, Appendix C.4, Theorem 3]. To provide measurable random variables for that statement, the Lean development additionally proves the following selector theorem. Let B be the compact parameter rectangle and $Q : \Omega \to B \to \mathbb{R}$ jointly measurable, with each section $Q(\omega, \cdot)$ continuous. The formalization constructs a measurable $\theta : \Omega \to B$ satisfying

$$Q(\omega, p) \le Q(\omega, \theta(\omega)) \quad \text{for every } \omega \text{ and } p \in B.$$

No uniqueness hypothesis is required.

ENN/Fit/Selection.lean:43--75

```
structure Select.Argmax
    (Ω : Type*) [MeasurableSpace Ω]
    (B : Fit.Box) (Q : Ω → Fit.region B → ℝ) where
  toFun : Ω → Fit.region B
  measurable_fn : Measurable toFun
  maximizes : ∀ ω, IsMaxOn (Q ω) Set.univ (toFun ω)

noncomputable def Select.argmax
    (B : Fit.Box) (Q : Ω → Fit.region B → ℝ)
    (hQ : Measurable (Function.uncurry Q))
    (hcont : ∀ ω, Continuous (Q ω)) :
    Select.Argmax Ω B Q
```

The proof constructs the measurable choice through successive subdivisions of the rectangle. On each compact cell, continuity identifies the maximum with the supremum over a fixed countable dense set. This makes the cell's maximum value measurable (`ENN.Select.max_measurable`).

Divide a cell into four closed children. At least one child retains its maximum; choose the first such child in a fixed order. Comparing measurable maximum values makes this choice measurable (`ENN.Select.step`). Repeating the choice produces nested maximizing cells with side lengths tending to zero (`ENN.Select.path`). Their centers converge to a single point. Continuity ensures that the limit attains the original maximum (`ENN.Select.limit_max`), and the pointwise limit of the measurable centers is measurable (`ENN.Select.limit_measurable`). The mathlib appendix gives the precise supremum, choice, and limit lemmas used here.

ENN/Select/Value.lean:20--52

ENN/Select/Cover.lean:58--104

ENN/Select/Path.lean:16--94

ENN/Select/Limit.lean:15--175

The construction is classical and noncomputable. Separate declarations supply measurable approximate selectors and the unique-maximizer specialization.

Limitations

The Lean development proves the mathematical declarations listed in this document. It does not verify the Python predictor, the numerical fitting routine, or an end-to-end implementation path from raw experiment logs to fitted ENN predictions. The noisy consistency results assume the radius and squared-weight estimates used in the proof; they do not derive those estimates from a distributional model of raw observations. The random-neighbor theorem accepts measurable random neighbors and a square-integrable random radius, but it does not construct a measurable sorting procedure from raw data.

The PseudoBO counterexample is about the abstract acquisition framework: it shows that the baseline $m_n = \inf_z \mu_n(z)$ alone does not imply worthiness for an arbitrary surrogate pair. It is not a counterexample to the ENN conditional improvement theorem. For fitted parameters, the argmax theorem assumes empirical maximality and separation for the stated criterion; those premises are the interface between the Lean result and any concrete leave-one-out fitting objective.

Appendix correspondence

Theorem 1(i). `ENN.Exact.error_envelope` and `ENN.Exact.error_bound` prove the approximation bound for an unsampled query with a nonempty neighbor prefix [1, Appendix C.1, Theorem 1(i)].

ENN/ExactObservation.lean:50--77

Theorem 1(ii). `ENN.Exact.spacing` and `ENN.Exact.empty_ball` give the two-sided spacing and empty-ball bounds [1, Appendix C.1, Theorem 1(ii)].

ENN/NearestNeighbor.lean:376--431

Theorem 1(iii–iv). `ENN.Exact.consistency`, `ENN.Exact.local_consistency`, and `ENN.Exact.moving_sneb` establish the fixed- and moving-query limits on compact metric spaces [1, Appendix C.1, Theorem 1(iii–iv)].

ENN/ExactObservation.lean:260--506

ENN/NearestNeighbor.lean:468--481

Proposition 1. `ENN.Acq.eventually_positive`, `ENN.Acq.vanishes`, and `ENN.Acq.linear_nondominated` prove the two improvement clauses and nondominance of the linear scalarization [1, Appendix C.2, Proposition 1].

ENN/AcquisitionGeometry.lean:100--200

Theorem 2. `ENN.Noise.mse_ordered`, `ENN.Noise.mse_indep`, and `ENN.Noise.consistency_cond` prove the conditional MSE inequality and its consistency consequence. `ENN.Noise.rate` and `ENN.Noise.balanced` derive the stated rate from assumed radius and squared-weight bounds [1, Appendix C.3, Theorem 2].

ENN/OrderedWeightedKNN.lean:25--119

ENN/Noise/RandomDesign.lean:202--254

ENN/Noise/Consistency.lean:124--148

ENN/Noise/Rates.lean:39--130

Theorem 3. `ENN.Fit.weights_l1` and `ENN.Fit.predictor_l1` give the stability constants; `ENN.Fit.argmax_limit` and `ENN.Fit.param_argmax` give argmax consistency from uniform

convergence and separation; `ENN.Fit.fitted_prob` and `ENN.Fit.fitted_l2` transfer oracle prediction bounds to fitted parameters [1, Appendix C.4, Theorem 3].

`ENN/Fit/Stability.lean:327--433`

`ENN/Fit/Argmax.lean:40--121`

`ENN/Fit/Transfer.lean:133--164`

Measurable fitting. `ENN.Select.argmax` and `ENN.Fit.argmax_measurable` provide exact measurable selection under joint measurability and continuity of the objective, together with measurable prediction for fixed finite arrays. These hypotheses must be checked for the fitting objective used in Theorem 3 [1, Appendix C.4, Theorem 3].

`ENN/Fit/Selection.lean:43--129`

Mathlib dependencies

The excerpts below identify the mathlib results used in the preceding proofs. Each citation links to the declaration's source lines [4]. An ellipsis omits surrounding arguments or proof steps.

A.1 Neighbor ordering

Sorting must preserve both distance order and a deterministic rule for ties. The distance–index keys are distinct because their second components are distinct. Mathlib's `Finset.orderEmbOfFin` enumerates this finite ordered set in increasing order. Projecting to the first coordinate gives the ordered radii.

`ENN/NearestNeighbor.lean:53--56,96--108`

```
noncomputable def Neighbor.ordered_key ... (j : Fin n) :
    ℝ ×ₗ Fin n :=
  (Neighbor.keys data query).orderEmbOfFin
    (Neighbor.keys_card data query) j

theorem Neighbor.radius_monotone ... :
    Monotone (Neighbor.radius data query) := by
  intro i j hij
  have hkey : ordered_key data query i ≤
      ordered_key data query j :=
    Neighbor.key_monotone data query hij
  have hfst := Prod.Lex.monotone_fst _ _ hkey
  rwa [Neighbor.key_radius, Neighbor.key_radius] at hfst
```

Mathlib: `Finset.orderEmbOfFin` (lines 194–211).

```
def orderEmbOfFin (s : Finset α) {k : ℕ}
    (h : s.card = k) : Fin k ↪o α :=
  (orderIsoOfFin s h).toOrderEmbedding.trans
    (OrderEmbedding.subtype _)

theorem orderEmbOfFin_apply ... :
    s.orderEmbOfFin h i =
      s.sort[i]'(by rw [length_sort, h]; exact i.2) := rfl
```

Mathlib: `Prod.Lex.monotone_fst` (lines 99–109).

```
theorem monotone_fst [Preorder α] [LE β]
    (t c : α ×ₗ β) (h : t ≤ c) :
    (ofLex t).1 ≤ (ofLex c).1 := by
  cases toLex_le_toLex.mp h with
  | inl h' => exact h'.le
  | inr h' => exact h'.1.le
```

The order embedding proves monotonicity of the keys. The second lemma proves that their first coordinates remain ordered, even when a comparison is decided by the index. This proves `ENN.Neighbor.radius_monotone`, which is used in the spacing and approximation bounds.

A.2 Local consistency

The local-consistency proof uses compactness to bound the function's range and continuity to control observations near the limiting query. The remaining task is to choose an index after which both the query displacement and the distance to the selected sample are sufficiently small.

`ENN/ExactObservation.lean:295--321`

```
rcases Metric.isBounded_range_iff.mp
    ((isCompact_range hf).isBounded) with (C, hC)
rw [Metric.tendsto_atTop]
intro ε hε
...
obtain (Nq, hNq) := Metric.tendsto_atTop.1 hq r hr
obtain (Nd, hNd) := Metric.tendsto_atTop.1 hd
  (min r (ε * r / (4 * B * K))) hηpos
```

Mathlib: `isCompact_range` (lines 961–963), `Metric.isBounded_range_iff` (lines 227–229), and `Metric.tendsto_atTop` (lines 911–924).

```
theorem isCompact_range [CompactSpace X]
    {f : X → Y} (hf : Continuous f) :
    IsCompact (range f)

theorem isBounded_range_iff {f : β → α} :
    IsBounded (range f) ↔
      ∃ C, ∀ x y, dist (f x) (f y) ≤ C

theorem tendsto_atTop ... :
    Tendsto u atTop (nhds a) ↔
      ∀ ε > 0, ∃ N, ∀ n ≥ N, dist (u n) a < ε
```

Compactness of the range and `Metric.isBounded_range_iff` give a constant bounding all differences of function values. This controls the error of distant observations. The calls to `Metric.tendsto_atTop` then produce indices `Nq` and `Nd` for the two convergence hypotheses. Beyond their maximum, both distance estimates hold. The ENN proof chooses their tolerances so that the near and far contributions together fit within the prescribed error.

A.3 Conditional centering

The weights may be random, but are measurable with respect to the conditioning σ-algebra. They can therefore be factored out of conditional expectations, provided the products are integrable. Applying this fact to each summand proves that a weighted sum of conditionally centered noises is conditionally centered.

`ENN/Noise/Moments.lean:30--45`

```
μ[∑ i, fun ω ↦ w i ω * ε i ω | m]
    =ᵐ[μ] ∑ i, μ[fun ω ↦ w i ω * ε i ω | m] := by
  simpa using condExp_finsetSum
    (s := Finset.univ)
    (f := fun i ω ↦ w i ω * ε i ω)
    (hf := fun i _ ↦ hterm_int i) m
_ =ᵐ[μ] ∑ i, w i * μ[ε i | m] := by
  refine eventuallyEq_sum fun i _ ↦ ?_
  exact condExp_mul_of_stronglyMeasurable_left
    (hw_meas i) (hterm_int i)
    ((hε2 i).integrable one_le_two)
```

Mathlib: `condExp_finsetSum` (lines 303–310).

```
theorem condExp_finsetSum ...
    (hf : ∀ i ∈ s, Integrable (f i) μ)
    (m : MeasurableSpace α) :
    μ[∑ i ∈ s, f i | m] =ᵐ[μ]
      ∑ i ∈ s, μ[f i | m]
```

Mathlib: `condExp_mul_of_stronglyMeasurable_left` (lines 245–252).

```
theorem condExp_mul_of_stronglyMeasurable_left
    (hf : StronglyMeasurable[m] f)
    (hfg : Integrable (f * g) μ)
    (hg : Integrable g μ) :
    μ[f * g | m] =ᵐ[μ] f * μ[g | m]
```

The first call distributes conditional expectation over the finite sum. The second factors out a weight using `hw_meas`, integrability of the weighted term, and integrability of the noise. Substituting the assumed zero conditional means proves `ENN.Noise.noise_mean`. The second-moment argument uses the same linearity and measurability properties after expanding the square.

A.4 Conditional orthogonality

Conditional independence and conditional centering imply that distinct noises have zero conditional cross moment. To prove this implication, the formalization uses mathlib's representation of conditional expectation as integration against a conditional-probability kernel. The theorem `ENN.Noise.condExp_mul` assumes a standard Borel sample space, a finite measure, measurable factors, and integrability of both factors and their product.

`ENN/Noise/Independence.lean:24--50`

```
theorem Noise.condExp_mul ...
    (hfg : CondIndepFun m hm f g μ) :
    condExp m μ (fun ω ↦ f ω * g ω) =ᵐ[μ]
      fun ω ↦ condExp m μ f ω * condExp m μ g ω := by
  have hmap := ae_eq_of_ae_eq_trim
    ((condIndepFun_iff_map_prod_eq_prod_map_map
      (mΩ := mΩ) hf hg).mp hfg)
  have hprod_cond :=
    condExp_ae_eq_integral_condExpKernel
      (mΩ := mΩ) hm hmul_int
  ...
  exact hindep.integral_mul_eq_mul_integral
    hf.aestronglyMeasurable hg.aestronglyMeasurable
```

Mathlib: `CondIndepFun` (lines 145–161).

```
def CondIndepFun ... (f : Ω → β) (g : Ω → γ)
    (μ : Measure Ω) [IsFiniteMeasure μ] : Prop :=
  Kernel.IndepFun f g (condExpKernel μ m') (μ.trim hm')
```

Mathlib: `condExp_ae_eq_integral_condExpKernel` (lines 243–260).

```
theorem condExp_ae_eq_integral_condExpKernel ...
    (hm : m ≤ mΩ) (hf_int : Integrable f μ) :
    μ[f | m] =ᵐ[μ]
      fun ω ↦ ∫ y, f y ∂condExpKernel μ m ω
```

Mathlib defines conditional independence using the conditional kernel. Under this kernel, independence factors the integral of a product into the product of the integrals. The conditional-expectation identity expresses this factorization in the notation used by the MSE theorem. If both conditional means are zero, their product is zero, proving the required conditional orthogonality.

A.5 Probability convergence

An almost-sure conditional MSE bound first gives an unconditional bound by integration. Markov's inequality for the squared error then bounds the probability of an error above any fixed positive threshold. These steps use the following two mathlib results.

`ENN/Noise/Consistency.lean:27--38,60--78`

```
(∫ ω, error ω ^ 2 ∂ν) =
    ∫ ω, ν[fun ω ↦ error ω ^ 2 | m] ω ∂ν := by
  exact (integral_condExp hm).symm
...
rw [tendstoInMeasure_iff_measureReal_dist]
...
have hmarkov := mul_meas_ge_le_integral_of_nonneg
  (f := fun ω ↦ (X n ω - target ω) ^ 2)
  (Eventually.of_forall fun _ ↦ sq_nonneg _)
  (hsq_int n) (δ ^ 2)
```

Mathlib: `integral_condExp` (lines 229–243).

```
theorem integral_condExp (hm : m ≤ m₀)
    [SigmaFinite (μ.trim hm)] :
    ∫ x, μ[f | m] x ∂μ = ∫ x, f x ∂μ
```

Mathlib: `mul_meas_ge_le_integral_of_nonneg` (lines 1129–1139).

```
theorem mul_meas_ge_le_integral_of_nonneg
    (hf_nonneg : 0 ≤ᵐ[μ] f) (hf_int : Integrable f μ)
    (ε : ℝ) :
    ε * μ.real {x | ε ≤ f x} ≤ ∫ x, f x ∂μ
```

The first identity removes the conditional expectation after integration. In the second inequality, the ENN proof takes the nonnegative function to be squared prediction error and the threshold to be δ^2. Thus an MSE tending to zero forces the measure of the event $|X_n - \text{target}| \geq \delta$ to tend to zero for every $\delta > 0$. This is the metric characterization of `TendstoInMeasure` (lines 57–68).

A.6 Argmax bounds

If separation holds and the uniform approximation error is less than $\eta/3$, the deterministic argmax lemma bounds the parameter distance by δ. Consequently, the event that this distance is at least δ is contained in the union of the two exceptional events below.

`ENN/Fit/Argmax.lean:71--90`

```
μ {ω | δ ≤ distance (hat n ω) (star n ω)}
    ≤ μ ({ω | ¬ Separated ... δ η} ∪
          {ω | η / 3 ≤ Δ n ω}) := by
  apply measure_mono
  intro ω hω
  ...
_ ≤ μ {ω | ¬ Separated ... δ η} +
      μ {ω | η / 3 ≤ Δ n ω} :=
  measure_union_le _ _
```

Mathlib: `measure_union_le` (lines 88–89).

```
theorem measure_union_le (s t : Set α) :
    μ (s ∪ t) ≤ μ s + μ t := by
  simpa [union_eq_iUnion] using
    measure_iUnion_fintype_le μ (cond · s t)
```

The ENN lemma proves the set inclusion. Mathlib's `measure_mono` bounds the measure by that of the union, and `measure_union_le` bounds the union by the sum. The separation and uniform-convergence hypotheses make both summands tend to zero.

A.7 Measurable selection

The subdivision proof requires measurability of the maximum value on each cell, of the chosen child, and of the limiting point. The three mathlib calls below correspond to those operations.

ENN/Select/Value.lean:37

ENN/Select/Cover.lean:68--75

ENN/Select/Limit.lean:36--52

```
have hsup_meas : Measurable
    (fun ω ↦ iSup fun n ↦ Q ω (seq n)) :=
  Measurable.iSup hseq_m

have hfind_m : Measurable
    (fun ω ↦ sel (Nat.find (hex ω)) ω) :=
  Measurable.find (f := sel) (p := fun n ω ↦ good n ω)
    (fun _ ↦ measurable_const) hgood_m hex

exact measurable_of_tendsto_metrizable hf hlim
```

Mathlib: `Measurable.iSup` (lines 908–915), `Measurable.find` (lines 509–513), and `measurable_of_tendsto_metrizable` (lines 51–54).

```
protected theorem Measurable.iSup ...
    (hf : ∀ i, Measurable (f i)) :
    Measurable (fun b ↦ iSup fun i ↦ f i b)

theorem Measurable.find ...
    (hf : ∀ n, Measurable (f n))
    (hp : ∀ n, MeasurableSet {x | p n x})
    (h : ∀ x, ∃ n, p n x) :
    Measurable fun x ↦ f (Nat.find (h x)) x

theorem measurable_of_tendsto_metrizable ...
    (hf : ∀ i, Measurable (f i))
    (lim : Tendsto f atTop (nhds g)) : Measurable g
```

For the first call, continuity identifies the cell maximum with the supremum along the chosen dense sequence. For the second, `hgood_m` proves measurability of the event that a child retains the maximum, and `hex` proves that such a child exists. The last call applies after the ENN proof has established convergence of the measurable cell centers. Attainment of the maximum is proved separately by continuity; it is not a conclusion of the measurable-limit theorem.

References

[1] Mehul Bafna, Siddhant Anand Jadhav, and David Sweet. Taking the GP out of the loop. *arXiv preprint arXiv:2506.12818v3*, 2026. https://doi.org/10.48550/arXiv.2506.12818.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[3] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. https://doi.org/10.1145/3372885.3373824.

[4] The mathlib Community. *mathlib4: The Mathematical Library of Lean 4*. Software repository, 2026. https://github.com/leanprover-community/mathlib4.

[5] Charles J. Stone. Consistent nonparametric regression. *The Annals of Statistics*, 5(4):595–620, 1977. https://doi.org/10.1214/aos/1176343886.

[6] Haoxian Chen and Henry Lam. Pseudo-Bayesian optimization. *arXiv preprint arXiv:2310.09766*, 2025. https://doi.org/10.48550/arXiv.2310.09766.